ROBOTICS INTEGRATION / GROUP PROJECT 1

Real-Time Desktop Object Detection

with YOLOv8, Jetson Orin NX, and ROS2

DATASET
1,000 images

FORMAL TEST
85.00% accuracy

FULL-LOOP SPEED
22.20 FPS mean

ROS2 OUTPUT
/qyy/detections

Author	**Yongyue Qi**
Repository	github.com/ccx8kg5qpd-debug/robotics-group-project-1
Platform	NVIDIA Jetson Orin NX / ROS2 Humble
Report date	31 August 2026
Document	Individually designed LaTeX report

Contents

1 Project Overview

This project implements a two-class desktop object detector for bottle and mouse. A YOLOv8n model was trained on a 1,000-image dataset, deployed on an NVIDIA Jetson Orin NX, and integrated with ROS2. The real-time application draws bounding boxes, class names, confidence values, and a smoothed full-loop frame rate. Structured detections are published as JSON through a ROS2 topic.

> The final formal test contains exactly 20 chronologically selected Jetson frames. Seventeen frames are fully correct and three contain a missed mouse, giving 85.00% frame-level accuracy. The measured mean speed is 22.20 FPS. Both results exceed the course requirements of 80% accuracy and 5 FPS.

1.1 Objectives and acceptance criteria

Table 1: Project acceptance criteria and verified status

Criterion	Requirement	Verified result
Object classes	At least two	bottle and mouse
Formal test	At least 20 instances	20 frames
Recognition accuracy	At least 80%	85.00%
Jetson speed	At least 5 FPS	16.7–25.8 FPS
ROS2 integration	Publish detections	/qyy/detections
Saved evidence	Results and errors	17 correct and 4 retained error examples

2 Hardware and Software Environment

Table 2: Verified training and deployment environment

Component	Configuration
Training computer	Apple Mac with Metal Performance Shaders (MPS)
Training framework	Ultralytics YOLOv8n
Deployment computer	NVIDIA Jetson Orin NX
Operating system	Ubuntu 22.04.5 LTS
JetPack / L4T	JetPack 6.2.1 / L4T R36.4.4
Python	3.10.12
CUDA	Approximately 12.6 in the deployed environment
ROS2	Humble
Camera	USB camera opened through cv2.VideoCapture(0)

The Jetson environment used an ARM64 PyTorch build compatible with JetPack 6 and CUDA 12.6. The missing libcudss.so.0 dependency was resolved by installing libcudss0-cuda-12 and adding /usr/lib/aarch64-linux-gnu/libcudss/12 to LD_LIBRARY_PATH. The deployed environment was verified with ROS2, Ultralytics, and torch.cuda.is_available() == True.

3 Dataset Construction

3.1 Data composition

The final dataset contains 1,000 images. Thirty images were self-captured with an iPhone in bottle-only, mouse-only, and combined desktop scenes. The remaining 970 images were selected from COCO 2017. Their bounding boxes came from the official COCO instance annotations and were converted to YOLO format.

An early auto-labelling utility was used only to assist inspection of the 30 self-captured images. It did not generate the official labels for the 970 public images. The COCO category mapping was bottle = 39 and mouse = 64; the final YOLO class mapping is 0 = bottle and 1 = mouse.

Table 3: Final dataset split and bounding-box counts

Split	Images	Self-captured	Public	Bottle boxes	Mouse boxes
Train	800	20	780	2,358	696
Validation	100	5	95	227	82
Test	100	5	95	297	95
Total	1,000	30	970	2,882	873

3.2 Source and licence records

Each public image is accompanied by its source URL, COCO identifier, licence name, licence URL, and SHA-256 value in dataset/source_info/source_manifest.csv. COCO images may carry different Creative Commons licences, so the project does not describe the complete public subset as having one uniform licence. Public data were used for course teaching and research; the accompanying source manifest should be consulted before any redistribution.

4 Dataset Quality Assurance

The final dataset was checked programmatically for image-label correspondence, empty labels, unsupported class identifiers, malformed rows, non-numeric coordinates, coordinates outside the normalised range, non-positive width or height, bounding boxes extending beyond the image, damaged images, and exact SHA-256 duplicates across splits.

Table 4: Dataset validation summary

Check	Result
Images / label files	1,000 / 1,000
Missing or orphan labels	0
Empty labels	0
Invalid class identifiers	0
Malformed or out-of-range coordinates	0
Damaged images	0
Exact cross-split duplicates	0

5 Model Training

YOLOv8n was selected for its compact size and suitability for real-time embedded inference. The model was trained for 100 epochs with 640-pixel inputs using Apple MPS.

```
yolo detect train \
  model=yolov8n.pt \
  data=final_dataset/data.yaml \
  epochs=100 \
  imgsz=640 \
  device=mps
```

The final row of results.csv records 14,421.7 seconds of accumulated training time, or approximately 4.006 hours. The selected best.pt file is 6,244,266 bytes.

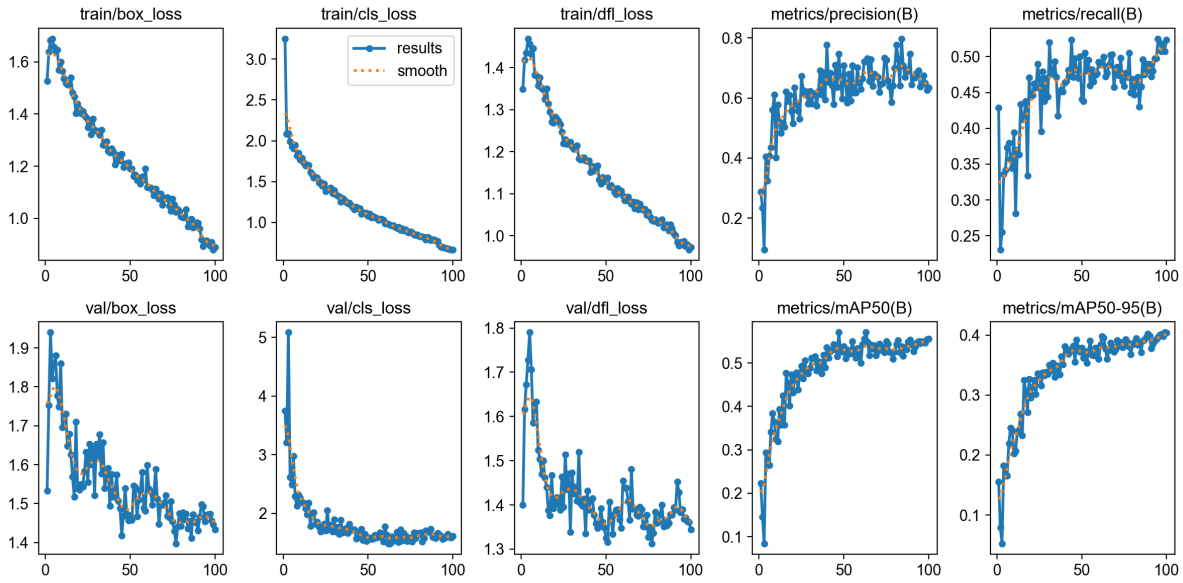


Figure 1: YOLOv8n training and validation curves over 100 epochs

6 Model Evaluation

The saved best.pt model was re-evaluated on the 100-image validation split. The all-class mAP50 is 0.555 and mAP50-95 is 0.405. Mouse detection is stronger than bottle detection in this validation set.

Table 5: Validation metrics for best.pt

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
All	100	309	0.676	0.488	0.555	0.405
Bottle	63	227	0.592	0.379	0.418	0.276
Mouse	62	82	0.760	0.598	0.691	0.533

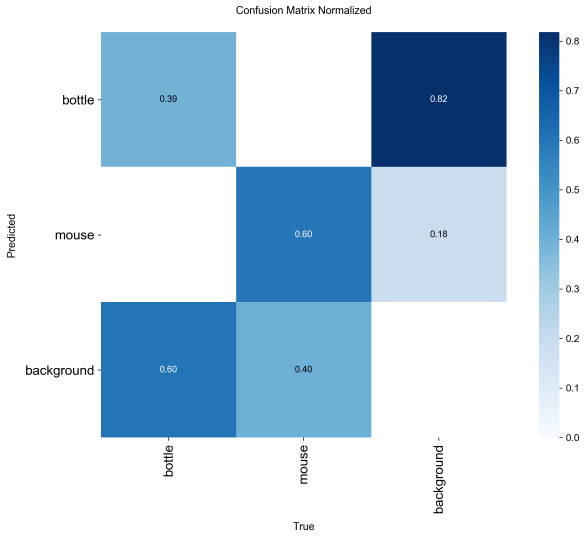


Figure 2: Normalised confusion matrix from model training

7 Jetson Real-Time Detection

The deployment directory on the Jetson was /home/nvidia/qyy_object_detection. Both programs load best.pt, use camera source 0, select CUDA device 0, and apply the final confidence threshold $\mathrm{conf} = 0.70$. The threshold was increased from the early 0.25 setting after keyboard regions produced false mouse detections at approximately 0.26 and, in one later observation, approximately 0.63. This was an inference-threshold adjustment rather than model retraining.

The real-time program performs the following sequence:

1. capture a frame from the USB camera;
2. run YOLO inference on the Jetson GPU;
3. draw class, confidence, and bounding-box overlays;
4. calculate a 30-frame moving average of complete loop intervals;
5. display the annotated frame and respond to keyboard input;
6. save correct cases with s, error cases with e, or quit with q.

8 ROS2 Integration

The ROS2 implementation creates node qyy_yolo_detector and publishes std_msgs/msg/String messages on /qyy/detections. Each message contains the current FPS and a list of detections with class name, confidence, and [x1, y1, x2, y2] pixel coordinates.

```
{
  "fps": 18.2,
  "detections": [
    {
      "class": "bottle",
      "confidence": 0.93,
      "bbox": [100, 100, 300, 400]
    }
  ]
}
```

The numbers above illustrate the message schema and are not formal test measurements. ROS2 operation was verified with ros2 topic list and continuous output from ros2 topic echo /qyy/detections.

9 Formal 20-Frame Jetson Test

The final test uses the first 20 saved frames from the 31 August Jetson session in chronological order. A frame is considered correct only if every visible bottle and mouse is detected with the correct class. Seventeen frames are fully correct and three contain one missed mouse. No visible object was assigned the wrong class.

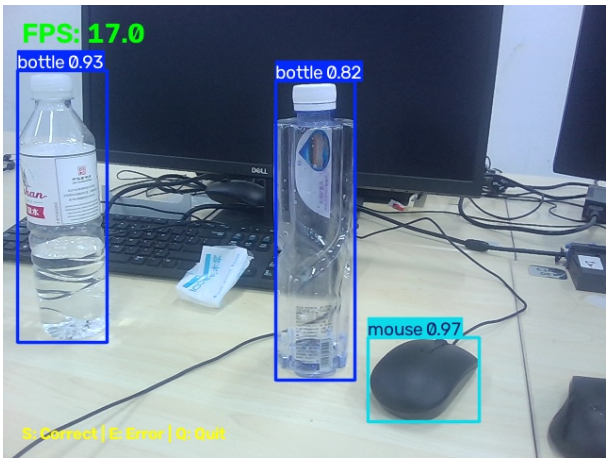
Table 6: Formal test summary

Frames	Correct	Failed	Accuracy	Requirement
20	17	3	85.00%	At least 80%

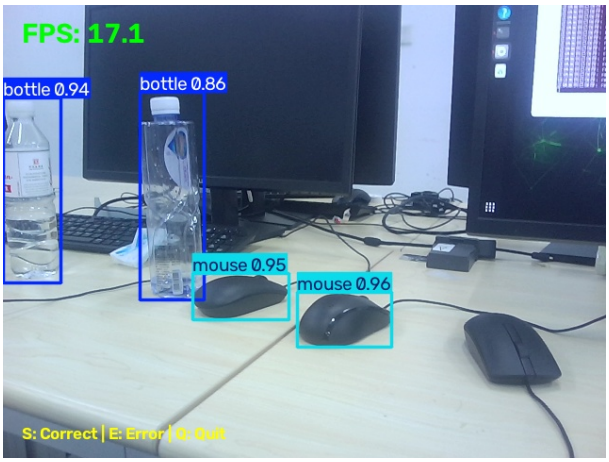
The saved full-loop FPS values range from 16.7 to 25.8, with an arithmetic mean of 22.20 FPS. Every formal test frame exceeds the 5 FPS requirement.

Table 7: Full-loop speed summary

Metric	Minimum	Mean	Maximum	Requirement
FPS	16.7	22.20	25.8	At least 5



Correct multi-class frame: two bottles and one mouse detected at 17.0 FPS.



Failed frame: two bottles and two mice detected, with the rightmost mouse missed.

Figure 3: Representative evidence from the final Jetson session

10 Error Analysis

All three formal failures are partial missed detections of a mouse. The detected objects retain the correct class labels, so these frames are false-negative cases rather than wrong-class predictions. During visual review, correct_20260831_215645_674393.jpg was found to show three physical mice but only two boxes. It was therefore reclassified as an error in the final test evidence without modifying the original captured file.

Figure 4: Reclassified case: the centre mouse is visible but has no detection box

An additional saved frame, error_20260831_215718_819397.jpg, is retained as supplementary evidence but excluded from the 20-frame calculation. It shows the same failure mode: correctly classified bottles

and mice with one mouse missed.

11 Demonstration Video

The final source recording is IMG_9884.MOV. Direct inspection reports a portrait resolution of 2160 by 3840 pixels, approximately 119.945 FPS, 8,389 frames, a duration of approximately 69.94 seconds, and a file size of 860,570,164 bytes. Sampled frames confirm live mouse detection, multi-mouse scenes, bottle-and-mouse scenes, bounding boxes, labels, confidence values, and displayed FPS above 5.

For the course package, the source is converted to a more manageable 1080 by 1920, 30 FPS copy while the original MOV remains unchanged. The public GitHub repository contains video metadata rather than the large media file.

12 Version Control and Repository

The project is maintained in the personal GitHub repository:

github.com/ccx8kg5qpd-debug/robotics-group-project-1

Commits are separated by meaningful project stages, including dataset documentation, Jetson and ROS2 programs, training and test evidence, English documentation, and the final report. The full public-data image set and large video file are not duplicated in the public repository; the source manifest and dataset validation evidence are retained.

```
$ git log --reverse --date=iso-local --pretty=...
repository: robotics-group-project-1    branch: main

b010bc1  2026-08-29 09:15:54 -0700  ccx8kg5qpd-debug
  docs: 添加项目说明与数据来源记录
f4da6e6  2026-08-29 09:15:54 -0700  ccx8kg5qpd-debug
  feat: 添加Jetson实时检测与ROS2发布程序
ec1e6a0  2026-08-29 09:15:54 -0700  ccx8kg5qpd-debug
  results: 添加训练评估与22次正式测试记录
ac4628e  2026-08-29 20:45:09 -0700  ccx8kg5qpd-debug
  report: 添加个人LaTeX模板与实验报告
afd6324  2026-08-30 00:00:01 -0700  ccx8kg5qpd-debug
  docs: 精简报告中的内部整理说明
df7fb01  2026-08-31 09:00:44 -0700  ccx8kg5qpd-debug
    results: refresh final Jetson test evidence
b85acda  2026-08-31 09:02:22 -0700  ccx8kg5qpd-debug
    docs: update English project documentation
```

Figure 5: Incremental Git commit history for the personal repository

13 Conclusion

The project completes the full workflow from dataset construction and quality assurance to YOLOv8 training, Jetson GPU deployment, real-time visualisation, evidence capture, and ROS2 publication. The final dataset contains 1,000 images, the selected model achieves validation mAP50-95 of 0.405, the 20-frame Jetson test reaches 85.00% accuracy, and the complete detection loop averages 22.20 FPS. The measured accuracy and speed both satisfy the assignment requirements. The retained missed-detection cases also identify a clear direction for future work: add more varied multi-object desktop scenes and difficult mouse viewpoints before further fine-tuning.

A Formal Test Table

Table 8: Chronological 20-frame formal test record

No.	Ground truth	Prediction	FPS	Correct
1	mouse	mouse	16.7	Yes
2	mouse x2	mouse x2	16.8	Yes
3	mouse	mouse	18.6	Yes
4	mouse x2	mouse x2	22.6	Yes
5	mouse x2	mouse x2	22.3	Yes
6	mouse x2	mouse x2	22.3	Yes
7	mouse x2	mouse x2	24.9	Yes
8	mouse x2	mouse x2	25.3	Yes
9	mouse	mouse	25.5	Yes
10	mouse x2	mouse x2	25.6	Yes
11	mouse	mouse	25.5	Yes
12	mouse x3	mouse x3	25.0	Yes
13	mouse x3	mouse x3	25.1	Yes
14	mouse x3	mouse x3	25.0	Yes
15	mouse x3	mouse x3	21.3	Yes
16	mouse x3	mouse x3	24.5	Yes
17	mouse x3	mouse x2	25.8	No
18	bottle x2 + mouse	bottle x2 + mouse	17.0	Yes
19	bottle x2 + mouse x3	bottle x2 + mouse x2	17.1	No
20	bottle x2 + mouse x2	bottle x2 + mouse	17.0	No